## 2026-7-14 Linux进程--命令行参数


- 1. 命令行参数如何进入进程
  - 1.1 main 函数可以接收参数
    - 1.1.1 argc 与 argv 的基本含义
    - 1.1.2 参数表以 NULL 结束
    - 1.1.3 参数属于一次进程启动
  - 1.2 shell 怎样生成命令行参数表
    - 1.2.1 按语法把命令行拆成参数
    - 1.2.2 参数从 shell 交给程序
    - 1.2.3 程序名和路径不是一回事
  - 1.3 命令行参数的作用
    - 1.3.1 让同一个程序执行不同功能
    - 1.3.2 参数也是进程任务的一部分
    - 1.3.3 库函数与 main 参数
    - 1.3.4 VS2022 也可以设置命令行参数
  - 1.4 文件参数与输入重定向
    - 1.4.1 cat file.txt：文件名是普通参数
    - 1.4.2 cat < file.txt：shell 完成输入重定向
    - 1.4.3 两种方式为什么结果相似
- 2. 从 PATH 理解环境变量
  - 2.1 为什么执行命令有时必须带路径
    - 2.1.1 ./ 是明确的相对路径
    - 2.1.2 PATH 决定去哪里搜索命令
    - 2.1.3 如何安全地修改 PATH
  - 2.2 环境变量的基本含义
    - 2.2.1 环境变量是进程启动时获得的环境参数
    - 2.2.2 环境变量保存在进程内存中
    - 2.2.3 环境变量的“全局”是向下继承
  - 2.3 常见环境变量
    - 2.3.1 PATH、HOME 和 SHELL
    - 2.3.2 HISTSIZE、OLDPWD 和 SSH_TTY
- 3. shell 与环境变量的来源
  - 3.1 shell 启动时加载配置
    - 3.1.1 新终端通常会启动新的 shell 进程
    - 3.1.2 bash 启动时从哪里获得环境
    - 3.1.3 临时修改与永久配置

- 3.2 环境从父进程传给子进程
  - 3.2.1 fork 与 exec 之间的继承链路
  - 3.2.2 继承不等于所有进程共享同一份内存
- 4. 查看、创建和删除环境变量
  - 4.1 常用命令
    - 4.1.1 echo、env、set 和 export
    - 4.1.2 本地变量与环境变量
    - 4.1.3 PATH 就是已经导出的环境变量
  - 4.2 内建命令为什么不受 PATH 限制
    - 4.2.1 普通命令与内建命令
    - 4.2.2 清空 PATH 的实验
    - 4.2.3 内建命令为什么能访问本地变量
- 5. 在 C/C++ 程序中读取环境变量
  - 5.1 环境变量表的形状
    - 5.1.1 环境变量是字符串指针数组
    - 5.1.2 main 的第三个参数
    - 5.1.3 全局变量 environ
  - 5.2 使用 getenv 按名字读取
    - 5.2.1 getenv 的基本用法
    - 5.2.2 为什么 getenv 更适合正常业务
  - 5.3 用环境变量观察进程继承
    - 5.3.1 在父 shell 中导出变量
    - 5.3.2 未导出的变量通常读不到
    - 5.3.3 子进程修改不会影响父进程
- 6. 环境变量的整体理解
  - 6.1 从命令查找到进程继承
  - 6.2 容易混淆的边界
  - 6.3 自检问题


**本篇范围**

前置阅读：2026-7-7 Linux进程--基础概念、2026-7-12 Linux进程--理解。本篇先整理 Linux 命令行参数如何由 shell 组织并传入 `main()`，再整理环境变量：它是什么、为什么能被继承、如何查看和修改，以及 C/C++ 程序如何读取。命令行参数表 `argv` 与环境变量表 `envp` 是两份不同的进程启动数据。

## 1. 命令行参数如何进入进程

### 1.1 main 函数可以接收参数

#### 1.1.1 argc 与 argv 的基本含义

C/C++ 程序的入口可以从简单形式：

```
int main(void)
```

扩展为：

```
int main(int argc, char *argv[])
```

其中：

- `argc` 表示参数个数（argument count）；
- `argv` 表示参数向量（argument vector），本质是字符串指针数组。

示例：

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);

    for (int i = 0; i < argc; ++i) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    return 0;
}
```

若程序名为 `code`，执行：

```
./code a b c d
```

参数表可以画成：

```
argc = 5

argv[0] -> "./code"
argv[1] -> "a"
argv[2] -> "b"
argv[3] -> "c"
argv[4] -> "d"
argv[5] -> NULL
```

`argv[0]` 通常指向程序被调用时使用的名称，例如 `./code`。`argv[1]` 到 `argv[argc - 1]` 才是用户附加的参数。

### 1.1.2 参数表以 NULL 结束

对于正常的 C 运行环境：

- `argc` 至少为 1；
- `argv[0]` 通常存在，用来表示程序名；
- `argv[argc]` 是空指针 `NULL`，作为参数表结束标记。

因此 `argc` 不是“额外参数的个数”，而是包含程序名在内的参数总数。

即使把参数写成一个字符，例如 `-n` 或 `a`，它们在 `argv` 中也仍然是字符串，而不是独立的 `char`。如果需要数字，程序必须自行把字符串转换为整数。

### 1.1.3 参数属于一次进程启动

命令行参数是本次程序启动的数据。一个程序被启动多次，每次都可以得到不同的 `argc` 和 `argv`。

子进程不会自动获得父进程的命令行参数。父进程若希望子进程看到某些值，需要在创建子进程或执行目标程序时显式传递；环境变量则具有另一套继承规则。

| 数据 | 入口 | 主要作用 | 传递特点 |
| --- | --- | --- | --- |
| 命令行参数 | `argc`、`argv` | 选择本次操作的参数、选项、文件 | 通常只影响本次程序调用 |

| 数据 | 入口 | 主要作用 | 传递特点 |
| --- | --- | --- | --- |
| 环境变量 | `envp` / `environ` | 提供长期运行配置 | 可从父进程继承 |

## 1.2 shell 怎样生成命令行参数表

### 1.2.1 按语法把命令行拆成参数

用户在终端输入：

```
./code a b c d
```

shell 需要先理解命令行的语法，再把程序名和各个参数整理成字符串表。

课堂常把这个过程简化为“shell 以空格分割命令行”。实际 shell 解析还会处理引号、转义和变量展开。例如：

```
./code "hello world" next
```

这里 `"hello world"` 虽然中间有空格，但被双引号括起来，仍然是一个参数：

```
argc = 3
argv[1] = "hello world"
argv[2] = "next"
```

所以更准确的说法是：shell 按自己的词法规则拆分命令行，而不是简单地遇到空格就无条件切开。

### 1.2.2 参数从 shell 交给程序

一次外部程序启动大致经过：

```
用户输入命令行
    ↓
shell 解析并形成 argv
    ↓
```

```
创建子进程，执行目标程序
    ↓
C 运行环境调用 main(argc, argv, ...)
```

启动程序时，操作系统需要得到：

- 要执行哪个程序；
- 参数表 `argv`；
- 环境表 `envp`。

在 Linux 的程序装载接口中，这三类信息对应 `execve(path, argv, envp)`。因此命令行参数和环境变量虽然可能同时出现在一个进程的启动信息中，但它们是两份独立的数据。

#### 1.2.3 程序名和路径不是一回事

`argv[0]` 是程序收到的“调用名称”，通常等同于用户输入的程序路径。它不是可以依赖的稳定文件路径：

- 通过不同路径启动程序，`argv[0]` 可能不同；
- 调用者也可以人为传入不同的名称；
- 若程序要打开自身所在文件，不能简单认为 `argv[0]` 已经是可以直接使用的绝对路径。

理解 `argv[0]` 时，应把它看成程序名称参数，而不是操作系统为程序保存的永久身份。

### 1.3 命令行参数的作用

#### 1.3.1 让同一个程序执行不同功能

命令行参数最常见的用途是让同一个可执行文件根据参数执行不同操作：

```
./tool add 10 20
./tool sub 10 20
./tool --help
./tool input.txt
```

这里第一项 `add`、`sub` 可以看作子命令，`--help` 可以看作选项，`input.txt` 可以看作操作对象。

因此命令行参数是程序外部接口的一部分：

```
同一份程序代码
    + 不同参数
    = 不同执行行为
```

Linux 中大量命令的单短选项、长选项和子命令，最终都要落到 `argv` 的解析上。

#### 1.3.2 参数也是进程任务的一部分

程序被启动后，会根据参数决定打开哪个文件、连接哪个地址、采用哪种模式，或者显示哪段帮助信息。

同样一个程序文件可以形成多个进程，这些进程可能执行相同代码，却因为参数不同而承担不同任务。参数不会变成新的代码，而是作为数据影响程序的执行路径。

#### 1.3.3 库函数与 main 参数

`main()` 是一个 C 程序的入口约定，不是所有函数都能接收命令行参数。库函数被调用时，拿不到某个“全局命令行参数表”并自动替调用者选择功能。

因此可复用库通常接收普通参数：

```c
int process_file(const char *path, int verbose);
```

由 `main()` 负责解析命令行参数，再把解析后的值传给库函数。这样库逻辑不必依赖程序怎样启动，也更方便测试和复用。

#### 1.3.4 VS2022 也可以设置命令行参数

命令行参数并不只存在于 Linux。在 VS2022 中，可以在项目属性中找到“调试”配置，填写 Command Arguments，以便调试时模拟传给程序的参数。

不同版本的位置名称可能略有差异，但本质都是把一组字符串作为 `argv` 传给目标程序。

### 1.4 文件参数与输入重定向

#### 1.4.1 `cat file.txt`：文件名是普通参数

执行：

```
cat file.txt
```

此时 `cat` 收到的参数表类似于：

```
argc = 2
argv[0] = "cat"
argv[1] = "file.txt"
argv[2] = NULL
```

`cat` 自己根据 `argv[1]` 打开文件，再把文件内容写到标准输出。

#### 1.4.2 `cat < file.txt`：shell 完成输入重定向

执行：

```
cat < file.txt
```

这时 `cat` 通常只收到程序名，没有文件参数：

```
argc = 1
argv[0] = "cat"
argv[1] = NULL
```

真正发生的事情是：shell 在启动 `cat` 前打开 `file.txt`，并把 `cat` 进程的标准输入指向这个文件。`cat` 不需要知道文件名，只需要从标准输入读取内容。

#### 1.4.3 两种方式为什么结果相似

| 写法 | 谁打开文件 | `cat` 收到什么 |
| --- | --- | --- |
| `cat file.txt` | `cat` | 文件名字符串 |
| `cat < file.txt` | shell | 只有程序名，标准输入已被重定向 |

两种写法最终都能让 `cat` 读取文件内容，但控制路径不同：

```
cat file.txt  -> 参数选择文件 -> cat 打开文件
cat < file.txt -> shell 打开文件 -> 标准输入交给 cat
```

因此不能把输入重定向简单解释成“把文件名作为命令行参数传给 cat”。

## 2. 从 PATH 理解环境变量

### 2.1 为什么执行命令有时必须带路径

#### 2.1.1 `./` 是明确的相对路径

假设当前目录下有一个自己编译的可执行文件：

```
ls -l mycommand
./mycommand
```

`./mycommand` 告诉 shell：要在当前目录 `.` 下寻找名为 `mycommand` 的文件。`./` 表示相对当前工作目录，不是某种“运行程序专用语法”。

如果直接输入：

```
mycommand
```

shell 会把它当作一个没有写明路径的命令名，然后去约定的目录中查找可执行文件。

#### 2.1.2 PATH 决定去哪里搜索命令

Linux 中的 `PATH` 环境变量保存一组目录，目录之间通常用冒号分隔。查看当前值：

```
printf '%s\n' "$PATH"
```

典型内容类似：

```
/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/user/.local/bin
```

当用户输入 `ls`、`gcc` 这类没有斜杠的命令名时，shell 会按 `PATH` 中的目录顺序查找可执行文件。

可以概括为：

> `PATH` 告诉 shell：没有显式给出路径时，应该到哪些目录中寻找命令。

查找顺序有意义：排在前面的目录先被搜索。因此如果不同目录中存在同名程序，实际执行的可能不是预想中的那个版本。

#### 2.1.3 如何安全地修改 PATH

把新目录追加到 PATH 末尾：

```
export PATH="$PATH:/opt/tools/bin"
```

不要写成：

```
PATH=/opt/tools/bin
```

后一种写法会覆盖原有搜索路径，导致 `ls`、`gcc` 等常见命令可能找不到。

如果真的把 PATH 清空，命令解释器仍可以执行内建命令，但需要 PATH 查找的外部程序会失败。可以临时用绝对路径恢复：

```
/usr/bin/ls
export PATH=/usr/local/bin:/usr/bin:/bin
```

实际路径应以本机环境为准。

### 2.2 环境变量的基本含义

#### 2.2.1 环境变量是进程启动时获得的环境参数

环境变量保存“变量名 = 变量值”形式的配置，例如：

```
PATH=/usr/local/bin:/usr/bin:/bin
HOME=/home/student
SHELL=/bin/bash
USER=student
```

应用程序可以读取这些配置来决定自己的运行行为。例如：

- 根据 PATH 查找外部程序；
- 根据 HOME 寻找用户主目录；
- 根据语言、编码或终端类型调整输出；
- 根据应用自定义变量选择开发、测试或生产配置。

环境变量不是 CPU 资源，也不是进程调度状态；它为进程提供一组启动时即可使用的配置数据。

### 2.2.2 环境变量保存在进程内存中

shell 启动后，会把环境变量加载到自己的内存中，以字符串指针数组的形式保存。

使用 `export` 修改环境变量时，改变的是当前 shell 进程的内存状态，并不会自动修改磁盘上的配置文件。关闭终端后，如果没有写入启动配置，这次临时修改就会丢失。

因此要区分：

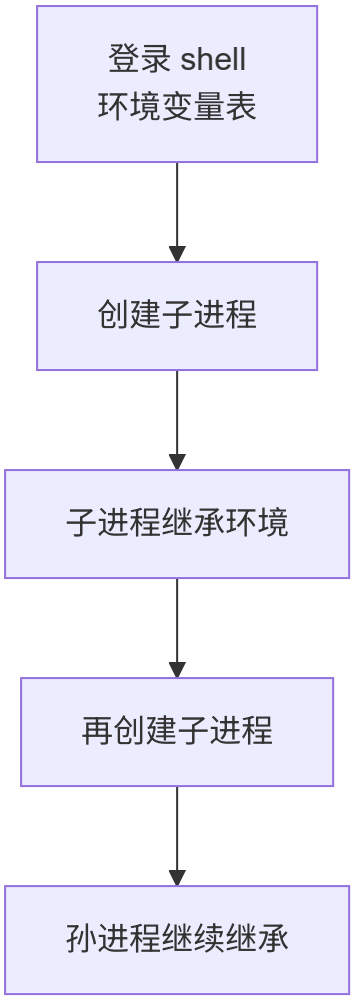
| 范围 | 生命周期 |
| --- | --- |
| 当前 shell 内存中的环境变量 | 当前 shell 进程存在期间 |
| 从 shell 创建的子进程 | 继承创建时父进程提供的环境 |
| 写入启动配置文件的环境变量 | 后续新 shell 启动时重新加载 |

### 2.2.3 环境变量的“全局”是向下继承

课堂常说环境变量具有全局属性，但这里的“全局”不是指所有 Linux 进程共享同一块内存。

更准确的理解是：

> 从某个父进程启动时继承下来的环境，会继续传给它的子进程；环境变量的影响范围通常沿进程树向下传递。




这带来三个结论：

- 子进程通常能看到父进程导出的环境变量；
- 子进程修改自己的环境，不会反向改变父进程；
- 两个无关进程不会仅因为都叫“环境变量”就共享同一份值。

### 2.3 常见环境变量

#### 2.3.1 PATH、HOME 和 SHELL

| 变量 | 作用 | 常见用途 |
| --- | --- | --- |
| `PATH` | 保存命令搜索目录 | 输入 `gcc` 时决定去哪些目录寻找 |
| `HOME` | 保存当前用户主目录 | `~` 展开、程序保存用户配置 |
| `SHELL` | 记录用户的默认登录 shell | 判断默认命令解释器 |

查看：

```
printf 'PATH=%s\n' "$PATH"
printf 'HOME=%s\n' "$HOME"
printf 'SHELL=%s\n' "$SHELL"
```

需要注意：`SHELL` 表示用户配置的默认登录 shell，不一定等于当前正在运行的 shell。要查看当前 shell，可以观察 `ps -p $$` 或 `$0`。

2.3.2 HISTSIZE、OLDPWD 和 SSH_TTY

| 变量 | 作用 |
| --- | --- |
| `HISTSIZE` | 控制当前 shell 历史命令列表中保留的条数，常见默认值为 1000 |
| `OLDPWD` | 记录上一次所在工作目录，`cd -` 会使用它 |
| `SSH_TTY` | 在 SSH 会话中记录终端设备路径，例如 `/dev/pts/0` |

例如：

```
echo "$HISTSIZE"
echo "$OLDPWD"
echo "$SSH_TTY"
```

`SSH_TTY` 只在 SSH 终端会话中常见，本地终端未必设置。要查看当前终端设备，直接运行 `tty` 更可靠。

## 3. shell 与环境变量的来源

### 3.1 shell 启动时加载配置

3.1.1 新终端通常会启动新的 shell 进程

每个交互式终端窗口通常都会有一个 shell 进程负责读取命令、解释命令并创建子进程。

可以观察当前 shell：

```
echo "shell pid: $$"
ps -p $$ -o pid,ppid,stat,cmd
```

也可以在另一个终端中观察 bash 进程：

```
while :; do
    ps axj | grep '[b]ash'
    sleep 1
done
```

不同终端窗口可能对应不同 shell 进程，因此它们的临时环境修改未必互相可见。

#### 3.1.2 bash 启动时从哪里获得环境

bash 的环境变量主要来自：

- 父进程传入的环境；
- 系统级启动配置文件；
- 用户主目录下的启动配置文件。

常见用户配置包括：

```
~/.bash_profile
~/.bashrc
```

不同启动方式读取的文件不同，简化理解如下：

- 交互式登录 shell 常读取 `~/.bash_profile`、`~/.bash_login` 或 `~/.profile`；
- 交互式非登录 shell 常读取 `~/.bashrc`；
- 很多发行版的 `~/.bash_profile` 会主动加载 `~/.bashrc`。

因此并不是修改任意一个文件都会立刻影响所有 shell。具体还要看 shell 是登录 shell 还是非登录 shell，以及配置文件中是否做了 `source`。

#### 3.1.3 临时修改与永久配置

当前终端的临时设置：

```
export MY_ENV="/opt/app/config"
```

这条命令只改变当前 shell 及其后续子进程的环境，不修改配置文件。

希望以后每次启动 shell 都生效，可以在合适的用户配置文件中加入同一行：

```
export MY_ENV="/opt/app/config"
```

修改后可使当前 shell 重新加载配置：

```
source ~/.bashrc
```

登录 shell 具体应加载哪个文件，需要结合本机配置判断。

### 3.2 环境从父进程传给子进程

#### 3.2.1 fork 与 exec 之间的继承链路

在典型 shell 执行外部命令的过程中：

```
shell 环境表
    ↓ fork 创建子进程
子进程获得父进程环境的继承关系
    ↓ exec 装载目标程序
目标程序从 envp/environ 获得环境
```

因此，用户没有在命令行中显式传入环境变量，程序仍然可以通过 `getenv()` 读取，因为 shell 在启动程序时已经把环境提供给了它。

#### 3.2.2 继承不等于所有进程共享同一份内存

父子进程在执行层面可能共享一些只读内容，并通过写时拷贝维护独立性。对于环境变量，应抓住以下语义：

- 子进程启动时获得一份环境值；
- 子进程后续修改自己的环境，不会自动修改父进程；
- 父进程修改当前环境，也不会反向改变已经运行的子进程。

所以“全局”描述的是继承链中的可见范围，不应理解成跨所有进程共享。

## 4. 查看、创建和删除环境变量

### 4.1 常用命令

#### 4.1.1 echo、env、set 和 export

| 命令 | 作用 |
| --- | --- |
| `echo "$NAME"` | 展开并输出指定变量的值 |
| `env` | 显示当前 shell 准备传给子进程的环境变量 |
| `set` | 显示 shell 变量、环境变量以及部分 shell 定义信息 |
| `export NAME` | 把已有的 shell 变量加入导出环境 |
| `export NAME=value` | 赋值并导出为环境变量 |
| `unset NAME` | 删除当前 shell 中的变量或环境项 |

示例：

```
export APP_MODE="development"
printf 'APP_MODE=%s\n' "$APP_MODE"
env | grep '^APP_MODE='
unset APP_MODE
```

删除后，当前 shell 不再导出该变量。若它在启动配置文件中重新定义，下次启动 shell 时还可能再次出现。

#### 4.1.2 本地变量与环境变量

在 bash 中为一个新名字直接赋值：

```
LOCAL_VALUE=100
echo "$LOCAL_VALUE"
env | grep '^LOCAL_VALUE='
```

通常可以观察到：`echo` 能显示它，`env` 看不到它。原因是它只是 shell 内部的本地变量，尚未导出为环境变量。

导出：

```
export LOCAL_VALUE
env | grep '^LOCAL_VALUE='
```

此时子进程才能通过环境接口获得它。

区别可以总结为：

| 类型 | 当前 shell 可见 | 子进程可继承 | 是否出现在 env 中 |
| --- | --- | --- | --- |
| shell 本地变量 | 是 | 否 | 通常否 |
| 已导出环境变量 | 是 | 是 | 是 |

本地变量适合保存 shell 内部临时数据和辅助脚本控制流程，不需要也不应该全部导出给子进程。

#### 4.1.3 PATH 就是已经导出的环境变量

PATH 通常是预先导出的环境变量，所以执行：

```
PATH="$PATH:/opt/tools/bin"
```

修改后，后续由 shell 创建的子进程通常也能看到新的 PATH。

但对新变量而言，仅写：

```
MY_ENV="hello"
```

并不会自动导出。需要执行：

```
export MY_ENV
```

或直接：

```
export MY_ENV="hello"
```

### 4.2 内建命令为什么不受 PATH 限制

#### 4.2.1 普通命令与内建命令

命令至少可以分成两类：

- **外部命令**：磁盘上存在可执行文件，shell 需要按 PATH 找到它并创建子进程；
- **内建命令**：由 shell 自己实现，在当前 shell 进程内部执行。

常见内建命令包括：

```
cd
export
unset
set
```

`echo`、`pwd` 等命令在 bash 中也可能以内建形式提供。可以使用：

```
type -a echo
type -a pwd
type -a ls
```

`type -a` 会列出同名命令的所有可用来源，便于判断某个名字到底来自 shell 内建还是磁盘文件。

#### 4.2.2 清空 PATH 的实验

课堂用下面的方式观察内建命令与外部命令的区别：

```
PATH=""
ls
echo "still works"
pwd
```

此时：

- `ls` 需要 shell 通过 PATH 找到外部程序，通常失败；
- `echo`、`pwd` 若是当前 shell 的内建命令，仍能工作；
- 可用 `/usr/bin/ls` 这样的绝对路径临时运行外部命令；
- 可执行 `export PATH=/usr/local/bin:/usr/bin:/bin` 恢复常见路径。

内建命令不需要按 PATH 查找，因此它也不必创建新的外部命令进程。

#### 4.2.3 内建命令为什么能访问本地变量

本地变量只存在于当前 shell 进程中。内建命令本身就是当前 shell 的一部分，所以可以直接读取本地变量；普通外部命令会形成子进程，只能接收已经导出的环境。

这解释了为什么 `echo "$LOCAL_VALUE"` 可以工作，而把同名变量交给一个独立 C 程序读取时，如果它没有导出，通常得到 `NULL`。

## 5. 在 C/C++ 程序中读取环境变量

### 5.1 环境变量表的形状

#### 5.1.1 环境变量是字符串指针数组

环境变量表可以抽象为：

```
char *envp[]
    ├── "PATH=/usr/local/bin:/usr/bin:/bin"
    ├── "HOME=/home/student"
    ├── "SHELL=/bin/bash"
    ├── ...
    └── NULL
```

每个元素指向一个以 `\0` 结尾的 `"NAME=value"` 字符串，数组最后用空指针标记结束。

程序遍历环境表时，可以一直读取到 `envp[i] == NULL` 为止。

### 5.1.2 main 的第三个参数

在 Linux 和常见 C/C++ 实现中，可以这样接收环境表：

```c
#include <stdio.h>

int main(int argc, char *argv[], char *envp[])
{
    (void)argc;
    (void)argv;

    for (int i = 0; envp[i] != NULL; ++i) {
        puts(envp[i]);
    }

    return 0;
}
```

这里的 `envp` 是程序启动时由操作系统和启动环境提供的环境表。它不是程序从零开始创建的。

### 5.1.3 全局变量 environ

glibc 还提供全局指针 `environ`，指向当前进程的环境表：

```c
#include <stdio.h>

extern char **environ;

int main(void)
{
    for (int i = 0; environ[i] != NULL; ++i) {
        puts(environ[i]);
    }

    return 0;
}
```

`environ` 适合需要遍历完整环境表的场景，但它通常需要由程序自行声明为 `extern`。

### 5.2 使用 getenv 按名字读取

#### 5.2.1 getenv 的基本用法

通常最方便的方式是：

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const char *path = getenv("PATH");
    const char *mode = getenv("APP_MODE");

    printf("PATH=%s\n", path != NULL ? path : "(not set)");
    printf("APP_MODE=%s\n", mode != NULL ? mode : "(not set)");

    return 0;
}
```

`getenv(name)` 会按变量名查找环境表：

```
找到 -> 返回指向变量值部分的指针
找不到 -> 返回 NULL
```

它返回的是环境表中值的地址，不是新复制出来的独立字符串。程序读取即可，不应随意修改环境字符串内容。

#### 5.2.2 为什么 getenv 更适合正常业务

三种方式的对比如下：

| 方式 | 特点 | 适用场景 |
| --- | --- | --- |
| `main` 的 `envp` | 启动时直接拿到完整表 | 教学、需要遍历全部环境 |
| `environ` | 全局入口，便于遍历 | 库或工具需要扫描完整环境 |

| 方式 | 特点 | 适用场景 |
| --- | --- | --- |
| `getenv` | 按名字读取，接口简单 | 正常业务按需获取配置 |

普通业务通常只关心少数几个变量，`getenv()` 比手工遍历整张表更清晰，也不依赖 main 的特殊参数形式。

### 5.3 用环境变量观察进程继承

#### 5.3.1 在父 shell 中导出变量

```
export TEST_ENV="hello-world"
./env_demo
```

如果 `env_demo` 调用 `getenv("TEST_ENV")`，通常能读到 `hello-world`。

#### 5.3.2 未导出的变量通常读不到

```
LOCAL_VALUE="only-in-shell"
./env_demo
```

如果程序中读取的是 `LOCAL_VALUE`，通常会得到 `NULL`，因为 `LOCAL_VALUE` 没有被导出到子进程的环境中。

#### 5.3.3 子进程修改不会影响父进程

子进程可以修改自己的环境，但这种变化只发生在子进程及其后续子进程中，不会回头修改父 shell。

因此环境变量的继承方向是：

```
父进程 -> 子进程 -> 孙进程
```

不是：

```
子进程修改 -> 父进程同步变化
```

## 6. 环境变量的整体理解

### 6.1 从命令查找到进程继承

```
shell 启动
    ↓
读取配置文件，形成内存中的环境表
    ↓
用户输入命令；shell 用 PATH 查找外部程序
    ↓
fork 创建子进程并传入环境
    ↓
exec 装载目标程序
    ↓
程序通过 envp / environ / getenv 读取环境
```

这条链路把几个看似独立的问题连在一起：

- 为什么 `./mycommand` 能运行，而 `mycommand` 可能找不到；
- 为什么环境变量能影响子进程；
- 为什么 `export` 后子进程才能读到本地变量；
- 为什么程序即使没有收到命令行参数，仍能取得 PATH、HOME 等配置。

### 6.2 容易混淆的边界

| 容易混淆的说法 | 更准确的理解 |
| --- | --- |
| 环境变量是所有进程共享的全局内存 | 它沿父子进程向下继承，不是跨所有进程共享 |
| 所有 shell 变量都是环境变量 | 只有导出的 shell 变量才进入环境表 |
| 修改环境变量会自动永久保存 | 默认只影响当前 shell 及其后续子进程 |
| `SHELL` 一定表示当前 shell | 它通常表示用户默认登录 shell |
| PATH 是内核直接提供命令查找 | 通常是 shell 使用 PATH 解析无路径命令 |
| 内建命令也要通过 PATH 找可执行文件 | 内建命令由当前 shell 自己执行 |

### 6.3 自检问题

**应该能够解释这些现象**

1. 为什么 `./mycommand` 不依赖 PATH，而 `mycommand` 会使用 PATH?
2. PATH 中已有路径时，为什么不能直接覆盖为单个新目录?
3. 环境变量的“全局属性”为什么不能理解为所有进程共享内存?
4. `LOCAL_VALUE=100` 与 `export LOCAL_VALUE=100` 有什么区别?
5. 为什么 `env` 看不到某个本地变量，而 `echo` 却能看到?
6. `echo "$LOCAL_VALUE"` 能工作，为什么独立 C 程序不导出就读不到?
7. `main` 的 `envp`、`environ` 和 `getenv()` 应如何选择?
8. 为什么修改 `.bashrc` 后，有的终端仍然没有立即生效?